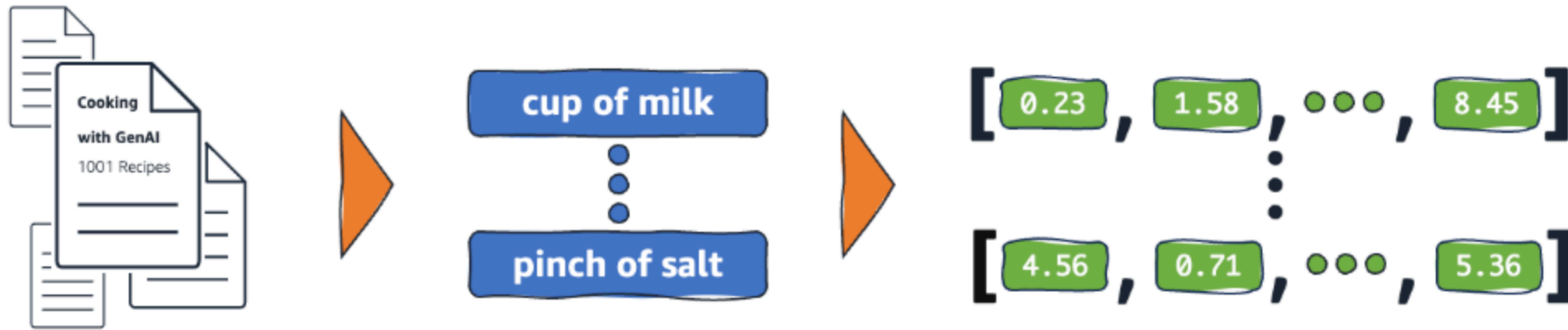


Node Embedding Algorithm — Neo4j GDS

Fast Random Projection (FastRP)

What is Embeddings



- <https://aws.amazon.com/what-is/embeddings-in-machine-learning/> (p.2~7)

Embeddings convert raw data into continuous values that ML models can interpret. Conventionally, ML models use one-hot encoding to map categorical variables into forms they can learn from. The encoding method divides each category into rows and columns and assigns them binary values. Consider the following categories of produce and their price.

Fruits	Price
Apple	5.00
Orange	7.00
Carrot	10.00

Representing the values with one-hot encoding results in the following table.

Apple	Orange	Pear	Price
1	0	0	5.00
0	1	0	7.00
0	0	1	10.00

The table is represented mathematically as vectors $[1,0,0,5.00]$, $[0,1,0,7.00]$, and $[0,0,1,10.00]$.

What are embedding models?

Embedding models are algorithms trained to encapsulate information into dense representations in a multi-dimensional space. Data scientists use embedding models to enable ML models to comprehend and reason with high-dimensional data. These are common embedding models used in ML applications.

Principal component analysis

Principal component analysis (PCA) is a dimensionality-reduction technique that reduces complex data types into low-dimensional vectors. It finds data points with similarities and compresses them into embedding vectors that reflect the original data. While PCA allows models to process raw data more efficiently, information loss may occur during processing.

Singular value decomposition

Singular value decomposition (SVD) is an embedding model that transforms a matrix into its singular matrices. The resulting matrices retain the original information while allowing models to better comprehend the semantic relationships of the data they represent. Data scientists use SVD to enable various ML tasks, including image compression, text classification, and recommendation.

Word2Vec

Word2Vec is an ML algorithm trained to associate words and represent them in the embedding space. Data scientists feed the Word2Vec model with massive textual datasets to enable natural language understanding. The model finds similarities in words by considering their context and semantic relationships.

There are two variants of Word2Vec—Continuous Bag of Words (CBOW) and Skip-gram. CBOW allows the model to predict a word from the given context, while Skip-gram derives the context from a given word. While Word2Vec is an effective word embedding technique, it cannot accurately distinguish contextual differences of the same word used to imply different meanings.

BERT

BERT is a transformer-based language model trained with massive datasets to understand languages like humans do. Like Word2Vec, BERT can create word embeddings from input data it was trained with. Additionally, BERT can differentiate contextual meanings of words when applied to different phrases. For example, BERT creates different embeddings for 'play' as in "I went to a play" and "I like to play."

Node embedding

Node embedding algorithms compute low-dimensional vector representations of nodes in a graph. These vectors, also called embeddings, can be used for machine learning.

FastRP

Fast Random Projection is a node embedding algorithm in the family of random projection algorithms. These algorithms are theoretically backed by the Johnson-Lindenstrauss lemma according to which one can project n vectors of arbitrary dimension into $O(\log(n))$ dimensions and still approximately preserve pairwise distances among the points. In fact, a linear projection chosen in a random way satisfies this property.

Such techniques therefore allow for aggressive dimensionality reduction while preserving most of the distance information. The FastRP algorithm operates on graphs, in which case we care about preserving similarity between nodes and their neighbors. This means that two nodes that have similar neighborhoods should be assigned similar embedding vectors. Conversely, two nodes that are not similar should not be assigned similar embedding vectors.

How It Works (1/2)

Algorithm Breakdown

1. Initialization (Very Sparse Random Projection)

Each node n is assigned an initial random vector $\mathbf{r}_n$ (or $\mathbf{e}_n^{(0)}$) using a very sparse random projection technique, creating low-dimensional vectors with mostly zeros.

2. Iterative Neighborhood Averaging

For each iteration $i = 1$ to k :

$$\mathbf{e}_n^{(i)} = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \mathbf{e}_m^{(i-1)}$$

where $\mathcal{N}(n)$ is the set of neighbors of node (n).

How It Works (2/2)

Algorithm Breakdown

3. Final Embedding Construction

The output embedding $\mathbf{e}_n$ is a weighted sum of normalized vectors from the initial projection and each iteration:

$$\mathbf{e}_n = w_0 \cdot \text{normalize}(\mathbf{r}_n) + \sum_{i=1}^k w_i \cdot \text{normalize}(\mathbf{e}_n^{(i)})$$

, where

- $\text{normalize}(\mathbf{v}) = \mathbf{v} / \|\mathbf{v}\|_2$ (*L2normalization*),
- w_0 is `nodeSelfInfluence` (weight for the original random vector),
- $w_1, w_2, \dots, w_k$ are the `iterationWeights`.

Key Properties

- **Efficiency:** Linear time in the number of edges, highly parallelizable.
- **Multi-scale:** Early iterations capture local structure; later ones capture global structure.
- **Sparsity:** The initial random projections are very sparse, reducing memory and computation.
- **Parameters:**
 - Embedding dimension (typically 100–256).
 - Number of iterations (k) (often 2–4).
 - Weights w_i (often decreasing, e.g., higher weight on closer neighborhoods).

Typical Implementation (Python)

<https://github.com/ekaratnida/social-network/blob/main/week4/FastRP.ipynb>

```
import numpy as np
import networkx as nx
from scipy.sparse import csr_matrix

def fast_rp(G, dim=128, k=3, node_self_influence=1.0, iteration_weights=None):
    if iteration_weights is None:
        iteration_weights = [1.0] * k

    n_nodes = G.number_of_nodes()
    node_idx = {node: i for i, node in enumerate(G.nodes())}

    # Very sparse random projection (initial vectors)
    # Example: ~1-2 non-zero entries per vector ( $\pm 1$ )
    R = np.zeros((n_nodes, dim))
    for i in range(n_nodes):
        # Very sparse: few random  $\pm 1$  entries
        nnz = max(1, int(dim * 0.01)) # adjust sparsity
        indices = np.random.choice(dim, nnz, replace=False)
        R[i, indices] = np.random.choice([-1.0, 1.0], nnz)

    embeddings = [R.copy()]

    # Iterative averaging
    A = nx.to_scipy_sparse_array(G, nodelist=list(G.nodes()), format='csr')
    D_inv = csr_matrix(1.0 / (A.sum(axis=1) + 1e-10)) # degree inverse

    current = R
    for i in range(k):
        # Average over neighbors:  $D^{-1} A$  * previous
        avg = D_inv @ A @ current
        embeddings.append(avg)
        current = avg

    # Final weighted normalized embedding
    final = node_self_influence * embeddings[0]
    for i, w in enumerate(iteration_weights, 1):
        final += w * embeddings[i]

    # L2 normalize each row
    norms = np.linalg.norm(final, axis=1, keepdims=True)
    final /= (norms + 1e-10)
```

Node properties

Most real-world graphs contain node properties which store information about the nodes and what they represent. The FastRP algorithm in the GDS library extends the original FastRP algorithm with a capability to take node properties into account. The resulting embeddings can therefore represent the graph more accurately.

The node property aware aspect of the algorithm is configured via the parameters `featureProperties` and `propertyRatio`. Each node property in `featureProperties` is associated with a randomly generated vector of dimension `propertyDimension`, where $\text{propertyDimension} = \text{embeddingDimension} * \text{propertyRatio}$. Each node is then initialized with a vector of size `embeddingDimension` formed by concatenation of two parts:

The first part is formed like in the standard FastRP algorithm, The second one is a linear combination of the property vectors, using the property values of the node as weights.

The algorithm then proceeds with the same logic as the FastRP algorithm. Therefore, the algorithm will output arrays of size embeddingDimension. The last propertyDimension coordinates in the embedding captures information about property values of nearby nodes (the "property part" below), and the remaining coordinates (embeddingDimension - propertyDimension of them; "topology part") captures information about nearby presence of nodes.

Embedding Structure (with Node Properties)

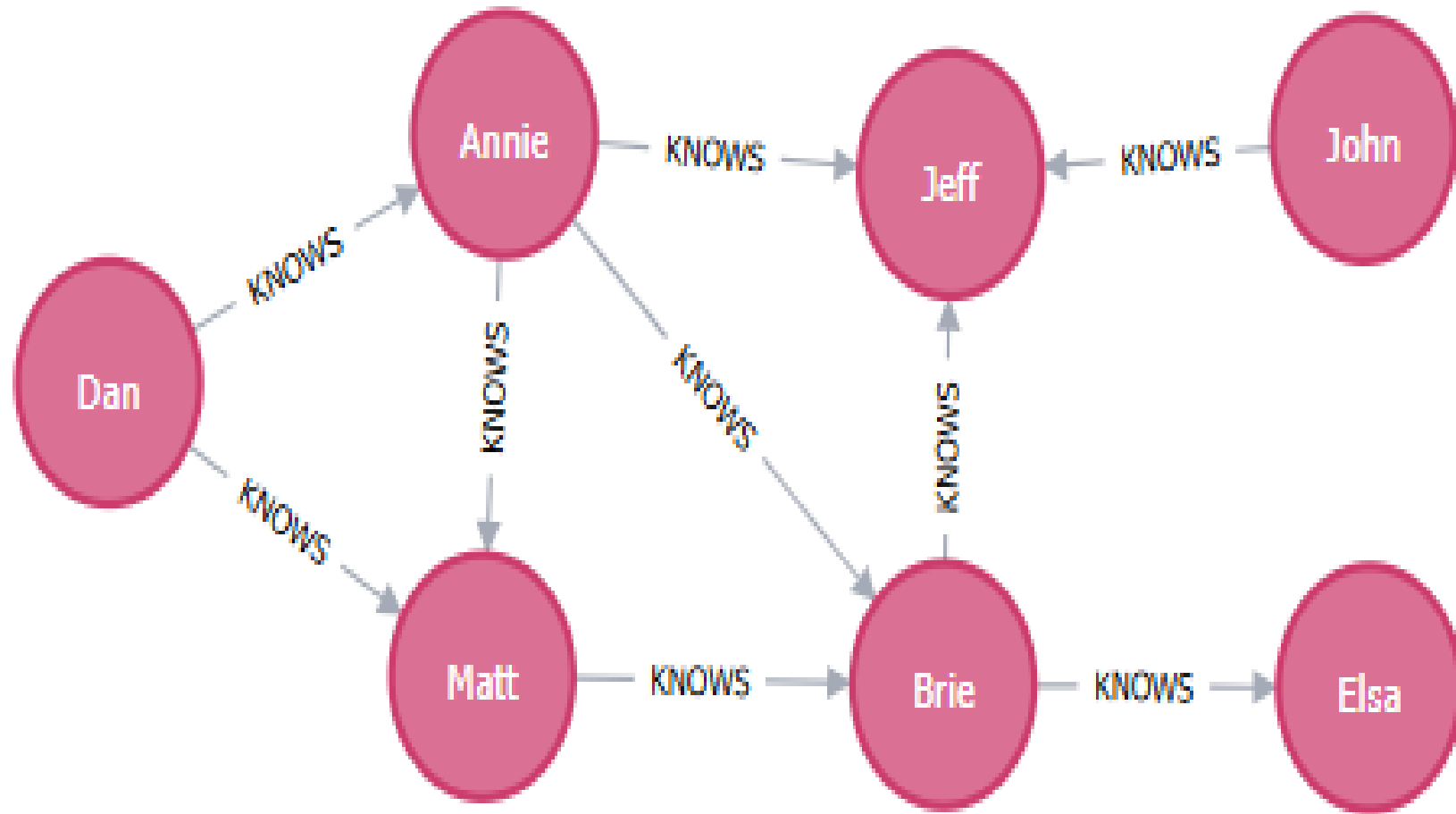
```
[0, 1, ... | ..., N - 1, N]
^^^^^^^^^^|^^^^^^^^^^
topology part | property part
              ^
            property ratio
```

- **Topology part** — captures neighborhood structure
- **Property part** — captures node property values of nearby nodes
- Controlled by `propertyRatio` and `featureProperties`

Execution Modes

Mode	Description
<code>stream</code>	Returns <code>nodeId</code> + <code>embedding</code> per node
<code>stats</code>	Returns <code>nodeCount</code> and timings
<code>mutate</code>	Writes embedding as node property to in-memory graph
<code>write</code>	Persists embedding as node property to Neo4j
<code>estimate</code>	Estimates memory requirements

Example



CREATE

```
(dan:Person {name: 'Dan', age: 18}),  
(annie:Person {name: 'Annie', age: 12}),  
(matt:Person {name: 'Matt', age: 22}),  
(jeff:Person {name: 'Jeff', age: 51}),  
(brie:Person {name: 'Brie', age: 45}),  
(elsa:Person {name: 'Elsa', age: 65}),  
(john:Person {name: 'John', age: 64}),
```

```
(dan)-[:KNOWS {weight: 1.0}]->(annie),  
(dan)-[:KNOWS {weight: 1.0}]->(matt),  
(annie)-[:KNOWS {weight: 1.0}]->(matt),  
(annie)-[:KNOWS {weight: 1.0}]->(jeff),  
(annie)-[:KNOWS {weight: 1.0}]->(brie),  
(matt)-[:KNOWS {weight: 3.5}]->(brie),  
(brie)-[:KNOWS {weight: 1.0}]->(elsa),  
(brie)-[:KNOWS {weight: 2.0}]->(jeff),  
(john)-[:KNOWS {weight: 1.0}]->(jeff);
```

The following statement will project a graph using a Cypher projection and store it in the graph catalog under the name 'persons'.

```
MATCH (source:Person)-[r:KNOWS]->(target:Person)
RETURN gds.graph.project(
  'persons',
  source,
  target,
  {
    sourceNodeProperties: source { .age },
    targetNodeProperties: target { .age },
    relationshipProperties: r { .weight }
  },
  { undirectedRelationshipTypes: ['*'] }
)
```

Stream example

```
CALL gds.fastRP.stream('persons',  
  {  
    embeddingDimension: 4,  
    randomSeed: 42  
  }  
)  
YIELD nodeId, embedding  
RETURN gds.util.asNode(nodeId).name as person, embedding  
ORDER BY person
```

Table 14. Results

person	embedding
"Annie"	[0.51714468, -0.4148067832, -0.5454565287, -1.741045475]
"Brie"	[0.4184039235, -0.4415202737, 0.2315290272, -1.5677155256]
"Dan"	[0.2612129152, -0.6138446331, -0.369674772, -1.7762401104]
"Elsa"	[0.5556756258, -0.3558300138, 0.308482945, -1.5653611422]
"Jeff"	[0.6856793165, -0.3247893453, -0.3811529875, -1.5765502453]
"John"	[1.0, -0.0890870914, -0.4454354346, -0.8908708692]
"Matt"	[0.2737978995, -0.4965225756, -0.3031099439, -1.8122189045]

Weight

```
CALL gds.fastRP.stream(  
  'persons',  
  {  
    embeddingDimension: 4,  
    randomSeed: 42,  
    relationshipWeightProperty: 'weight'  
  }  
)  
YIELD nodeId, embedding  
RETURN gds.util.asNode(nodeId).name as person, embedding  
ORDER BY person
```

Table 18. Results

person	embedding
"Annie"	[0.034561187, -0.2929389477, 0.0952546224, -1.9652962685]
"Brie"	[0.1023679227, -0.2991863489, 0.5466092229, -1.2881529331]
"Dan"	[-0.0909044892, -0.4465829134, 0.3275954127, -1.6877939701]
"Elsa"	[0.0776494294, -0.2621908784, 0.5610812902, -1.2880744934]
"Jeff"	[0.1686269641, -0.2775687575, 0.4166130424, -1.3728146553]
"John"	[0.5247224569, -0.045596078, 0.3423381448, -0.9119215012]
"Matt"	[0.0523263216, -0.3151839674, 0.4781413078, -1.4239065647]

Using Node properties as features

```
CALL gds.fastRP.stream('persons', {  
    randomSeed: 42,  
    embeddingDimension: 2,  
    propertyRatio: 1.0,  
    featureProperties: ['age'],  
    iterationWeights: [1.0]  
}) YIELD nodeId, embedding  
RETURN gds.util.asNode(nodeId).name AS person, embedding  
ORDER BY person
```

Table 19. Results

person	embedding
"Annie"	[0.0, -1.0]
"Brie"	[0.0, -0.9999999403953552]
"Dan"	[0.0, -1.0]
"Elsa"	[0.0, -1.0]
"Jeff"	[0.0, -1.0]
"John"	[0.0, -1.0]
"Matt"	[0.0, -0.9999999403953552]

Parameter Tuning Tips

1. **Embedding dimension** — 128-1024, 256 gives good results
2. **Iteration weights** — try [1, 1, 1] or [0, 1, 1] for 2-3 hops
3. **Normalization strength** — tune in [-1, 0]
4. **Property ratio** — use > 0 if node properties are informative
5. **Node self influence** — enable for graphs with isolated nodes
6. **Random seed** — set for reproducibility

Evaluate using a downstream ML task (link prediction, node classification).

Key Takeaways

- FastRP = **linear-time** node embeddings via random projection
- Encodes both **topology** and **node properties**
- Configurable neighborhood radius via `iterationWeights`
- Supports **weighted** and **directed** graphs
- Output: dense vectors ready for ML pipelines (KNN, classification, etc.)

Use case

- Product recommendations with kNN + FastRP

References

- [FastRP Documentation](#)
- Chen et al., *Fast and Accurate Network Embeddings via Very Sparse Random Projection* (2019) <https://arxiv.org/pdf/1908.11512>
- Achlioptas, *Database-friendly random projections* (2003) <https://www.sciencedirect.com/science/article/pii/S0022000003000254>

